

APPENDIX

Complete configuration files

Companion to the AWS Deployment Guide: EC2 vs ECS Fargate

Every file referenced in the guide, in full. Account IDs, domains and the GitHub repository are placeholders (search for **REPLACE**). Both Terraform directories passed **terraform validate** and a mocked **terraform plan** (Terraform 1.16.5, AWS provider 6.68.0); scripts passed **shellcheck**, workflows **actionlint**. They have not been applied to a real AWS account by the author of this guide.

Suggested .gitignore additions

```
.terraform/ · *.tfstate · *.tfstate.* · terraform.tfvars · tfplan · *.tfplan · crash.log (commit .terraform.lock.hcl)
```

Contents

A · Terraform files shared by both stacks	3
providers.tf	3
network.tf	3
security-groups.tf	5
load-balancer.tf	7
rds.tf	7
dns-acm.tf	8
frontend.tf	10
github-oidc.tf	11
monitoring.tf	12
B · EC2 stack (infrastructure/aws/ec2/)	15
versions.tf	15
variables.tf	15
terraform.tfvars.example	17
iam.tf	17
ec2.tf	19
templates/user-data.sh.tftpl	21
templates/ai-lab-api.service	23
scripts/deploy-release.sh	23
outputs.tf	24
C · ECS Fargate stack (infrastructure/aws/ecs/)	26
versions.tf	26
variables.tf	26
terraform.tfvars.example	28
iam.tf	29
ecr.tf	30
ecs-cluster.tf	30
ecs-task-definition.tf	31
ecs-service.tf	31
autoscaling.tf	32
outputs.tf	33
D · Container packaging	35
services/ai-engineering-api/Dockerfile	35
services/ai-engineering-api/.dockerignore	35
E · GitHub Actions workflows	36
.github/workflows/deploy-ecs.yml	36
.github/workflows/deploy-ec2.yml	38
F · Rendered JSON references (not committed; shown for learning)	41
ecs-task-definition.json	41
IAM: GitHub OIDC trust policy	41
IAM: ecs-tasks trust policy	41
IAM: execution role secrets policy	42
app-secret.example.json	42

A · Terraform files shared by both stacks

Identical copies live in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`.

`providers.tf` — Provider config, default tags, us-east-1 alias, shared locals.

`providers.tf` identical in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`

```
# providers.tf - which AWS region/account Terraform talks to.
# Default provider = the workload region. The aliased provider exists ONLY because
# CloudFront requires its ACM certificate to live in us-east-1.

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project      = var.project
      Environment  = var.environment
      ManagedBy    = "terraform"
    }
  }
}

provider "aws" {
  alias   = "us_east_1"
  region = "us-east-1"

  default_tags {
    tags = {
      Project      = var.project
      Environment  = var.environment
      ManagedBy    = "terraform"
    }
  }
}

data "aws_caller_identity" "current" {}

locals {
  name           = "${var.project}-${var.environment}" # ai-lab-prod
  app_fqdn       = "${var.app_subdomain}.${var.domain_name}" # app.example.com
  api_fqdn       = "${var.api_subdomain}.${var.domain_name}" # api.example.com
  account_id     = data.aws_caller_identity.current.account_id
}
```

network.tf — VPC, 3 subnet tiers × 2 AZs, IGW, NAT, routes, S3 gateway endpoint.**network.tf** identical in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`

```
# network.tf - VPC, subnets in 2 AZs, routing, NAT, free S3 gateway endpoint.
# public subnets : ALB + NAT gateway (route to internet gateway)
# private subnets : EC2 instances / Fargate tasks (outbound via NAT only)
# db subnets : RDS (no route to the internet at all)

data "aws_availability_zones" "available" {
  state = "available"
}

locals {
  azs          = slice(data.aws_availability_zones.available.names, 0, var.az_count)
  public_subnets = [for i in range(var.az_count) : cidrsubnet(var.vpc_cidr, 8, i)] # 10.20.0.0/24 ...
  private_subnets = [for i in range(var.az_count) : cidrsubnet(var.vpc_cidr, 8, i + 10)] # 10.20.10.0/24 ...
  db_subnets     = [for i in range(var.az_count) : cidrsubnet(var.vpc_cidr, 8, i + 20)] # 10.20.20.0/24 ...
  nat_count       = var.single_nat_gateway ? 1 : var.az_count
}

resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true
  tags            = { Name = "${local.name}-vpc" }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "${local.name}-igw" }
}

resource "aws_subnet" "public" {
  count          = var.az_count
  vpc_id         = aws_vpc.main.id
  cidr_block     = local.public_subnets[count.index]
  availability_zone = local.azs[count.index]
  map_public_ip_on_launch = false # nothing in here gets a public IP by accident
  tags           = { Name = "${local.name}-public-${local.azs[count.index]}", Tier = "public" }
}

resource "aws_subnet" "private" {
  count          = var.az_count
  vpc_id         = aws_vpc.main.id
  cidr_block     = local.private_subnets[count.index]
  availability_zone = local.azs[count.index]
  tags           = { Name = "${local.name}-private-${local.azs[count.index]}", Tier = "private" }
}

resource "aws_subnet" "db" {
  count          = var.az_count
  vpc_id         = aws_vpc.main.id
  cidr_block     = local.db_subnets[count.index]
  availability_zone = local.azs[count.index]
  tags           = { Name = "${local.name}-db-${local.azs[count.index]}", Tier = "db" }
}

resource "aws_eip" "nat" {
  count = local.nat_count
  domain = "vpc"
  tags   = { Name = "${local.name}-nat-eip-${count.index}" }
}

resource "aws_nat_gateway" "main" {
  count          = local.nat_count
  allocation_id = aws_eip.nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id
  tags          = { Name = "${local.name}-nat-${count.index}" }
  depends_on    = [aws_internet_gateway.main]
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }
  tags = { Name = "${local.name}-public-rt" }
}

resource "aws_route_table_association" "public" {
  count          = var.az_count
  subnet_id     = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

# One route table per AZ so each AZ can use its own NAT when single_nat_gateway = false.
resource "aws_route_table" "private" {
  count = var.az_count
  vpc_id = aws_vpc.main.id
  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main[var.single_nat_gateway ? 0 : count.index].id
  }
  tags = { Name = "${local.name}-private-rt-${local.azs[count.index]}" }
}
```

network.tf identical in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`

```
resource "aws_route_table_association" "private" {
  count          = var.az_count
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private[count.index].id
}

# DB route table: local VPC route only (implicit). No internet path in either direction.
resource "aws_route_table" "db" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "${local.name}-db-rt" }
}

resource "aws_route_table_association" "db" {
  count          = var.az_count
  subnet_id      = aws_subnet.db[count.index].id
  route_table_id = aws_route_table.db.id
}

# Gateway endpoint for S3: no hourly charge; keeps S3 traffic (artifacts, ECR image
# layers, OS packages) off the NAT gateway and its per-GB processing fee.
resource "aws_vpc_endpoint" "s3" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.${var.aws_region}.s3"
  vpc_endpoint_type = "Gateway"
  route_table_ids   = aws_route_table.private[*].id
  tags              = { Name = "${local.name}-s3-endpoint" }
}

# OPTIONAL: interface endpoints (each has an hourly + per-GB charge, per AZ) let private
# workloads reach AWS APIs without NAT. Add the ones you need, for example:
#   ECS/ECR : ecr.api, ecr.dkr, logs, secretsmanager
#   EC2/SSM : ssm, ssmmessages, ec2messages, logs, secretsmanager
# resource "aws_vpc_endpoint" "interface" {
#   for_each      = toset(["ecr.api", "ecr.dkr", "logs", "secretsmanager"])
#   vpc_id        = aws_vpc.main.id
#   service_name  = "com.amazonaws.${var.aws_region}.${each.key}"
#   vpc_endpoint_type = "Interface"
#   subnet_ids    = aws_subnet.private[*].id
#   security_group_ids = [aws_security_group.endpoints.id]
#   private_dns_enabled = true
# }
```

security-groups.tf — ALB → app → DB chain; explicit egress.**security-groups.tf** identical in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`

```
# security-groups.tf - the only permitted traffic paths:
# Internet --443/80--> ALB --api_port--> app (EC2 or Fargate) --5432--> RDS
# app --443--> AWS APIs / external HTTPS APIs (through NAT or VPC endpoints)
# Terraform removes the AWS default "allow all egress" rule, so every egress is explicit.

resource "aws_security_group" "alb" {
  name         = "${local.name}-alb-sg"
  description  = "Public entry point: HTTPS from the internet"
  vpc_id       = aws_vpc.main.id
  tags        = { Name = "${local.name}-alb-sg" }
}

resource "aws_vpc_security_group_ingress_rule" "alb_https" {
  security_group_id = aws_security_group.alb.id
  description       = "HTTPS from anywhere"
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "tcp"
  from_port         = 443
  to_port           = 443
}

resource "aws_vpc_security_group_ingress_rule" "alb_http" {
  security_group_id = aws_security_group.alb.id
  description       = "HTTP only to redirect to HTTPS"
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "tcp"
  from_port         = 80
  to_port           = 80
}

resource "aws_vpc_security_group_egress_rule" "alb_to_app" {
  security_group_id = aws_security_group.alb.id
  description       = "Forward requests and health checks to the API"
  referenced_security_group_id = aws_security_group.app.id
  ip_protocol       = "tcp"
  from_port         = var.api_port
  to_port           = var.api_port
}

# Used by EC2 instances (Approach A) or Fargate task ENIs (Approach B).
resource "aws_security_group" "app" {
  name         = "${local.name}-app-sg"
  description  = "API compute: reachable only from the ALB"
  vpc_id       = aws_vpc.main.id
  tags        = { Name = "${local.name}-app-sg" }
}

resource "aws_vpc_security_group_ingress_rule" "app_from_alb" {
  security_group_id = aws_security_group.app.id
  description       = "API port from the ALB only"
  referenced_security_group_id = aws_security_group.alb.id
  ip_protocol       = "tcp"
  from_port         = var.api_port
  to_port           = var.api_port
}

resource "aws_vpc_security_group_egress_rule" "app_https" {
  security_group_id = aws_security_group.app.id
  description       = "HTTPS to AWS APIs (ECR, S3, Secrets Manager, SSM, Logs) and external APIs"
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "tcp"
  from_port         = 443
  to_port           = 443
}

resource "aws_vpc_security_group_egress_rule" "app_to_db" {
  security_group_id = aws_security_group.app.id
  description       = "PostgreSQL to RDS"
  referenced_security_group_id = aws_security_group.db.id
  ip_protocol       = "tcp"
  from_port         = 5432
  to_port           = 5432
}

resource "aws_security_group" "db" {
  name         = "${local.name}-db-sg"
  description  = "RDS PostgreSQL: reachable only from the API"
  vpc_id       = aws_vpc.main.id
  tags        = { Name = "${local.name}-db-sg" }
}

resource "aws_vpc_security_group_ingress_rule" "db_from_app" {
  security_group_id = aws_security_group.db.id
  description       = "PostgreSQL from the API only"
  referenced_security_group_id = aws_security_group.app.id
  ip_protocol       = "tcp"
  from_port         = 5432
  to_port           = 5432
}

# No SSH (22) rule anywhere: shell access uses SSM Session Manager / ECS Exec over 443.
```

load-balancer.tf — ALB, target group (target type from local.lb_target_type), listeners.**load-balancer.tf** identical in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`

```
# load-balancer.tf - public ALB in 2 AZs. TLS terminates here; the ALB forwards plain
# HTTP to the API inside the VPC. local.lb_target_type is "instance" (EC2 stack)
# or "ip" (Fargate stack: every task has its own ENI/IP in awsvpc mode).

resource "aws_lb" "main" {
  name                = "${local.name}-alb"
  load_balancer_type = "application"
  internal            = false
  security_groups     = [aws_security_group.alb.id]
  subnets            = aws_subnet.public[*].id
  drop_invalid_header_fields = true
  idle_timeout        = 60
  enable_deletion_protection = true # set false (and apply) before terraform destroy
}

resource "aws_lb_target_group" "api" {
  name                = "${local.name}-api-tg"
  port                = var.api_port
  protocol             = "HTTP"
  vpc_id              = aws_vpc.main.id
  target_type         = local.lb_target_type
  deregistration_delay = 30 # seconds to drain in-flight requests on deploy/scale-in

  health_check {
    path            = var.health_check_path
    matcher         = "200"
    interval        = 15
    timeout         = 5
    healthy_threshold = 2
    unhealthy_threshold = 3
  }
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.main.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type = "redirect"
    redirect {
      port            = "443"
      protocol        = "HTTPS"
      status_code     = "HTTP_301"
    }
  }
}

resource "aws_lb_listener" "https" {
  load_balancer_arn = aws_lb.main.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn   = aws_acm_certificate_validation.api.certificate_arn
  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.api.arn
  }
}
```

rds.tf — PostgreSQL 17, Multi-AZ, RDS-managed secret, app-secret lookup.**rds.tf** identical in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`

```
# rds.tf - private, encrypted, Multi-AZ PostgreSQL 17. pgvector is available on
# RDS for PostgreSQL; the master user may run CREATE EXTENSION vector.
# manage_master_user_password = true -> RDS generates the password and stores it in
# Secrets Manager. The password is never written to Terraform state or to Git.

resource "aws_db_subnet_group" "main" {
  name       = "${local.name}-db-subnets"
  subnet_ids = aws_subnet.db[*].id
  tags       = { Name = "${local.name}-db-subnets" }
}

resource "aws_db_parameter_group" "main" {
  name       = "${local.name}-pg17"
  family     = "postgres17"

  parameter {
    name = "rds.force_ssl" # reject unencrypted client connections
    value = "1"
  }

  parameter {
    name = "log_min_duration_statement" # log statements slower than 1 s
    value = "1000"
  }
}

resource "aws_db_instance" "main" {
  identifier      = "${local.name}-db"
  engine          = "postgres"
  engine_version = var.db_engine_version
  instance_class  = var.db_instance_class

  allocated_storage      = var.db_allocated_storage
  max_allocated_storage   = var.db_max_allocated_storage # storage autoscaling ceiling
  storage_type            = "gp3"
  storage_encrypted       = true

  db_name          = var.db_name
  username          = var.db_username
  manage_master_user_password = true

  db_subnet_group_name   = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.db.id]
  parameter_group_name   = aws_db_parameter_group.main.name
  publicly_accessible    = false
  multi_az               = var.db_multi_az

  backup_retention_period = 7
  backup_window           = "03:00-04:00"
  maintenance_window      = "sun:04:30-sun:05:30"
  auto_minor_version_upgrade = true
  copy_tags_to_snapshot    = true

  enabled_cloudwatch_logs_exports = ["postgresql"]

  deletion_protection      = true
  skip_final_snapshot      = false
  final_snapshot_identifier = "${local.name}-db-final"
}

locals {
  # Secret JSON: {"username": "...", "password": "..."}
  db_secret_arn = aws_db_instance.main.master_user_secret[0].secret_arn
}

# Application secrets (API keys etc.) are created ONCE with the AWS CLI before the first
# apply, so their values never pass through Terraform state. Terraform only looks it up.
data "aws_secretsmanager_secret" "app" {
  name = "${var.project}/${var.environment}/api" # ai-lab/prod/api
}
```

dns-acm.tf — ACM certificates (2 regions), DNS validation, alias records.**dns-acm.tf** identical in infrastructure/aws/ec2/ and infrastructure/aws/ecs/

```

# dns-acm.tf - TLS certificates (validated through Route 53) and DNS aliases.
# api.example.com -> ALB (certificate in the workload region)
# app.example.com -> CloudFront (certificate MUST be in us-east-1)
# Assumes a public hosted zone for var.domain_name already exists in Route 53.

data "aws_route53_zone" "main" {
  name           = var.domain_name
  private_zone   = false
}

resource "aws_acm_certificate" "api" {
  domain_name       = local.api_fqdn
  validation_method = "DNS"
  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_acm_certificate" "app" {
  provider           = aws.us_east_1
  domain_name        = local.app_fqdn
  validation_method = "DNS"
  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_route53_record" "cert_validation" {
  for_each = {
    for dvo in concat(
      tolist(aws_acm_certificate.api.domain_validation_options),
      tolist(aws_acm_certificate.app.domain_validation_options)
    ) : dvo.domain_name => {
      name       = dvo.resource_record_name
      record     = dvo.resource_record_value
      type       = dvo.resource_record_type
    }
  }

  zone_id = data.aws_route53_zone.main.zone_id
  name     = each.value.name
  type     = each.value.type
  records  = [each.value.record]
  ttl      = 60
  allow_overwrite = true
}

resource "aws_acm_certificate_validation" "api" {
  certificate_arn       = aws_acm_certificate.api.arn
  validation_record_fqdns = [aws_route53_record.cert_validation[local.api_fqdn].fqdn]
}

resource "aws_acm_certificate_validation" "app" {
  provider           = aws.us_east_1
  certificate_arn     = aws_acm_certificate.app.arn
  validation_record_fqdns = [aws_route53_record.cert_validation[local.app_fqdn].fqdn]
}

resource "aws_route53_record" "api" {
  zone_id = data.aws_route53_zone.main.zone_id
  name     = local.api_fqdn
  type     = "A"
  alias {
    name           = aws_lb.main.dns_name
    zone_id        = aws_lb.main.zone_id
    evaluate_target_health = true
  }
}

resource "aws_route53_record" "app" {
  for_each = toset(["A", "AAAA"])
  zone_id = data.aws_route53_zone.main.zone_id
  name     = local.app_fqdn
  type     = each.key
  alias {
    name           = aws_cloudfront_distribution.frontend.domain_name
    zone_id        = aws_cloudfront_distribution.frontend.hosted_zone_id
    evaluate_target_health = false
  }
}

```

frontend.tf — Private S3, OAC, CloudFront with SPA fallback, bucket policy.**frontend.tf** identical in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`

```
# frontend.tf - React static hosting: PRIVATE S3 bucket -> CloudFront (OAC) -> HTTPS.
# Nobody can read the bucket directly; only this CloudFront distribution can.

resource "aws_s3_bucket" "frontend" {
  bucket = "${local.name}-frontend-${local.account_id}" # bucket names are global
}

resource "aws_s3_bucket_public_access_block" "frontend" {
  bucket                = aws_s3_bucket.frontend.id
  block_public_acls     = true
  block_public_policy    = true
  ignore_public_acls    = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_ownership_controls" "frontend" {
  bucket = aws_s3_bucket.frontend.id
  rule {
    object_ownership = "BucketOwnerEnforced" # disables ACLs
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "frontend" {
  bucket = aws_s3_bucket.frontend.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_cloudfront_origin_access_control" "frontend" {
  name                = "${local.name}-frontend-oac"
  description         = "CloudFront signs S3 requests with SigV4"
  origin_access_control_origin_type = "s3"
  signing_behavior    = "always"
  signing_protocol    = "sigv4"
}

data "aws_cloudfront_cache_policy" "caching_optimized" {
  name = "Managed-CachingOptimized"
}

data "aws_cloudfront_response_headers_policy" "security_headers" {
  name = "Managed-SecurityHeadersPolicy"
}

resource "aws_cloudfront_distribution" "frontend" {
  enabled            = true
  comment           = "${local.name} React SPA"
  aliases            = [local.app_fqdn]
  default_root_object = "index.html"
  is_ipv6_enabled    = true
  http_version       = "http2and3"
  price_class        = "PriceClass_100" # cheapest edge set; widen if users are global

  origin {
    origin_id           = "s3-frontend"
    domain_name         = aws_s3_bucket.frontend.bucket_regional_domain_name
    origin_access_control_id = aws_cloudfront_origin_access_control.frontend.id
  }

  default_cache_behavior {
    target_origin_id           = "s3-frontend"
    viewer_protocol_policy     = "redirect-to-https"
    allowed_methods            = ["GET", "HEAD"]
    cached_methods            = ["GET", "HEAD"]
    compress                   = true
    cache_policy_id            = data.aws_cloudfront_cache_policy.caching_optimized.id
    response_headers_policy_id = data.aws_cloudfront_response_headers_policy.security_headers.id
  }

  # SPA fallback: a deep link such as /orders/42 is not an S3 object. Without
  # s3:ListBucket S3 answers 403 (not 404), so map both to index.html with 200.
  custom_error_response {
    error_code           = 403
    response_code        = 200
    response_page_path   = "/index.html"
    error_caching_min_ttl = 10
  }

  custom_error_response {
    error_code           = 404
    response_code        = 200
    response_page_path   = "/index.html"
    error_caching_min_ttl = 10
  }

  restrictions {
    geo_restriction {
      restriction_type = "none"
    }
  }
}
```

frontend.tf identical in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`

```
viewer_certificate {
  acm_certificate_arn      = aws_acm_certificate_validation.app.certificate_arn
  ssl_support_method       = "sni-only"
  minimum_protocol_version = "TLSv1.2_2021"
}

data "aws_iam_policy_document" "frontend_bucket" {
  statement {
    sid      = "AllowCloudFrontOACRead"
    actions  = ["s3:GetObject"]
    resources = ["${aws_s3_bucket.frontend.arn}/*"]
    principals {
      type       = "Service"
      identifiers = ["cloudfront.amazonaws.com"]
    }
    condition {
      test      = "StringEquals"
      variable  = "AWS:SourceArn"
      values    = [aws_cloudfront_distribution.frontend.arn]
    }
  }
}

resource "aws_s3_bucket_policy" "frontend" {
  bucket    = aws_s3_bucket.frontend.id
  policy    = data.aws_iam_policy_document.frontend_bucket.json
  depends_on = [aws_s3_bucket_public_access_block.frontend]
}
```

github-oidc.tf — OIDC provider, deploy role trust, frontend publish policy.**github-oidc.tf** identical in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`

```
# github-oidc.tf - GitHub Actions gets SHORT-LIVED AWS credentials via OIDC.
# No AWS access keys are stored in GitHub. Only workflows running in the GitHub
# environment "production" of var.github_repo can assume this role.
# NOTE: an AWS account can hold only ONE provider for this URL. If it already exists,
# replace this resource with: data "aws_iam_openid_connect_provider" "github" { url = ... }

resource "aws_iam_openid_connect_provider" "github" {
  url      = "https://token.actions.githubusercontent.com"
  client_id_list = ["sts.amazonaws.com"]
}

data "aws_iam_policy_document" "github_assume" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    principals {
      type       = "Federated"
      identifiers = [aws_iam_openid_connect_provider.github.arn]
    }
    condition {
      test     = "StringEquals"
      variable = "token.actions.githubusercontent.com:aud"
      values   = ["sts.amazonaws.com"]
    }
    condition {
      test     = "StringEquals"
      variable = "token.actions.githubusercontent.com:sub"
      values   = ["repo:${var.github_repo}:environment:production"]
    }
  }
}

resource "aws_iam_role" "github_deploy" {
  name               = "${local.name}-github-deploy"
  assume_role_policy = data.aws_iam_policy_document.github_assume.json
  max_session_duration = 3600
}

# Permissions shared by both approaches: publish the React build and invalidate CloudFront.
data "aws_iam_policy_document" "github_frontend" {
  statement {
    sid      = "ListFrontendBucket"
    actions  = ["s3:ListBucket"]
    resources = [aws_s3_bucket.frontend.arn]
  }
  statement {
    sid      = "WriteFrontendObjects"
    actions  = ["s3:PutObject", "s3:DeleteObject"]
    resources = ["${aws_s3_bucket.frontend.arn}/*"]
  }
  statement {
    sid      = "InvalidateCdn"
    actions  = ["cloudfront:CreateInvalidation", "cloudfront:GetInvalidation"]
    resources = [aws_cloudfront_distribution.frontend.arn]
  }
}

resource "aws_iam_role_policy" "github_frontend" {
  name     = "frontend-deploy"
  role     = aws_iam_role.github_deploy.id
  policy   = data.aws_iam_policy_document.github_frontend.json
}
```

monitoring.tf — SNS topic and ALB/RDS alarms (illustrative thresholds).

monitoring.tf identical in `infrastructure/aws/ec2/` and `infrastructure/aws/ecs/`

```
# monitoring.tf - alarms that page someone. Thresholds are ILLUSTRATIVE starting points;
# tune them from load tests and real traffic.

resource "aws_sns_topic" "alerts" {
  name = "${local.name}-alerts"
}

resource "aws_sns_topic_subscription" "email" {
  count = var.alert_email == "" ? 0 : 1
  topic_arn = aws_sns_topic.alerts.arn
  protocol = "email" # the recipient must click the confirmation link
  endpoint = var.alert_email
}

locals {
  alb_dimensions = {
    LoadBalancer = aws_lb.main.arn_suffix
    TargetGroup    = aws_lb_target_group.api.arn_suffix
  }
}

resource "aws_cloudwatch_metric_alarm" "api_5xx" {
  alarm_name        = "${local.name}-api-5xx"
  alarm_description = "API returns 5xx responses"
  namespace         = "AWS/ApplicationELB"
  metric_name       = "HTTPCode_Target_5XX_Count"
  dimensions        = local.alb_dimensions
  statistic          = "Sum"
  period            = 60
  evaluation_periods = 3
  datapoints_to_alarm = 2
  threshold          = 10
  comparison_operator = "GreaterThanThreshold"
  treat_missing_data = "notBreaching"
  alarm_actions      = [aws_sns_topic.alerts.arn]
  ok_actions         = [aws_sns_topic.alerts.arn]
}

resource "aws_cloudwatch_metric_alarm" "api_latency_p95" {
  alarm_name        = "${local.name}-api-latency-p95"
  alarm_description = "p95 target response time above 1 second"
  namespace         = "AWS/ApplicationELB"
  metric_name       = "TargetResponseTime"
  dimensions        = local.alb_dimensions
  extended_statistic = "p95"
  period            = 60
  evaluation_periods = 5
  datapoints_to_alarm = 3
  threshold          = 1
  comparison_operator = "GreaterThanThreshold"
  treat_missing_data = "notBreaching"
  alarm_actions      = [aws_sns_topic.alerts.arn]
}

resource "aws_cloudwatch_metric_alarm" "api_unhealthy_hosts" {
  alarm_name        = "${local.name}-api-unhealthy-hosts"
  alarm_description = "At least one target failing ALB health checks"
  namespace         = "AWS/ApplicationELB"
  metric_name       = "UnHealthyHostCount"
  dimensions        = local.alb_dimensions
  statistic          = "Maximum"
  period            = 60
  evaluation_periods = 3
  threshold          = 0
  comparison_operator = "GreaterThanThreshold"
  treat_missing_data = "notBreaching"
  alarm_actions      = [aws_sns_topic.alerts.arn]
}

resource "aws_cloudwatch_metric_alarm" "db_cpu" {
  alarm_name        = "${local.name}-db-cpu"
  namespace         = "AWS/RDS"
  metric_name       = "CPUUtilization"
  dimensions        = { DBInstanceIdentifier = aws_db_instance.main.identifier }
  statistic          = "Average"
  period            = 300
  evaluation_periods = 3
  threshold          = 80
  comparison_operator = "GreaterThanThreshold"
  alarm_actions      = [aws_sns_topic.alerts.arn]
}

resource "aws_cloudwatch_metric_alarm" "db_connections" {
  alarm_name        = "${local.name}-db-connections"
  alarm_description = "Connection count approaching max_connections (set threshold from SHOW max_connections)"
  namespace         = "AWS/RDS"
  metric_name       = "DatabaseConnections"
  dimensions        = { DBInstanceIdentifier = aws_db_instance.main.identifier }
  statistic          = "Maximum"
  period            = 60
  evaluation_periods = 5
  threshold          = var.db_connections_alarm_threshold
  comparison_operator = "GreaterThanThreshold"
}
```

```
monitoring.tf identical in infrastructure/aws/ec2/ and infrastructure/aws/ecs/
  alarm_actions      = [aws_sns_topic.alerts.arn]
}

resource "aws_cloudwatch_metric_alarm" "db_free_storage" {
  alarm_name      = "${local.name}-db-free-storage"
  namespace       = "AWS/RDS"
  metric_name     = "FreeStorageSpace"
  dimensions      = { DBInstanceIdentifier = aws_db_instance.main.identifier }
  statistic       = "Minimum"
  period          = 300
  evaluation_periods = 1
  threshold       = 5368709120 # 5 GiB in bytes
  comparison_operator = "LessThanThreshold"
  alarm_actions    = [aws_sns_topic.alerts.arn]
}
```

B · EC2 stack (infrastructure/aws/ec2/)

versions.tf — Pins + S3 backend (key prod/ec2/...).

versions.tf infrastructure/aws/ec2/versions.tf

```
# versions.tf - pins Terraform + provider versions and stores state remotely.
# The state bucket is created ONCE by hand (see the bootstrap step); never commit state.

terraform {
  required_version = ">= 1.10.0" # S3 native state locking (use_lockfile)

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 6.0"
    }
  }

  backend "s3" {
    bucket = "ai-lab-tfstate-111122223333" # REPLACE: your state bucket (account ID suffix)
    key    = "prod/ec2/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    use_lockfile = true # lock file in S3; no DynamoDB table needed
  }
}
```

variables.tf — All inputs incl. ASG sizes and initial_release.**variables.tf** infrastructure/aws/ec2/variables.tf

```
# variables.tf - every input of the EC2 stack. Values go in terraform.tfvars.

variable "project" {
  description = "Short project name used as a prefix for every resource"
  type        = string
  default     = "ai-lab"
}

variable "environment" {
  description = "Environment name (prod, staging, ...)"
  type        = string
  default     = "prod"
}

variable "aws_region" {
  description = "Region for the VPC, ALB, compute and RDS"
  type        = string
  default     = "us-east-1"
}

variable "domain_name" {
  description = "Apex domain of an existing Route 53 public hosted zone"
  type        = string
}

variable "app_subdomain" {
  description = "Subdomain for the React app (CloudFront)"
  type        = string
  default     = "app"
}

variable "api_subdomain" {
  description = "Subdomain for the backend API (ALB)"
  type        = string
  default     = "api"
}

variable "github_repo" {
  description = "GitHub repository allowed to deploy, as OWNER/REPO"
  type        = string
}

variable "alert_email" {
  description = "Email for CloudWatch alarm notifications (empty = none)"
  type        = string
  default     = ""
}

# --- Network -----
variable "vpc_cidr" {
  type    = string
  default = "10.20.0.0/16"
}

variable "az_count" {
  description = "Number of Availability Zones (2 minimum for ALB and Multi-AZ RDS)"
  type        = number
  default     = 2
  validation {
    condition     = var.az_count >= 2 && var.az_count <= 3
    error_message = "az_count must be 2 or 3."
  }
}

variable "single_nat_gateway" {
  description = "true = one NAT (cheaper, AZ-level single point of failure); false = one per AZ"
  type        = bool
  default     = true
}

# --- API -----
variable "api_port" {
  type    = number
  default = 8080
}

variable "health_check_path" {
  type    = string
  default = "/actuator/health"
}

# --- Database -----
variable "db_engine_version" {
  type    = string
  default = "17"
}

variable "db_instance_class" {
  type    = string
  default = "db.t4g.medium"
}

variable "db_allocated_storage" {
```

variables.tf infrastructure/aws/ec2/variables.tf

```

type      = number
default   = 50
}

variable "db_max_allocated_storage" {
  type    = number
  default = 200
}

variable "db_multi_az" {
  type    = bool
  default = true
}

variable "db_name" {
  type    = string
  default = "ai_lab"
}

variable "db_username" {
  type    = string
  default = "ai_lab_admin"
}

variable "db_connections_alarm_threshold" {
  description = "Alarm when connections exceed this; derive from SHOW max_connections"
  type        = number
  default     = 300
}

# --- EC2 Auto Scaling (ILLUSTRATIVE sizes: confirm with load tests) -----
variable "instance_type" {
  type    = string
  default = "t3.medium"
}

variable "asg_min_size" {
  type    = number
  default = 2
}

variable "asg_desired_capacity" {
  type    = number
  default = 2
}

variable "asg_max_size" {
  type    = number
  default = 10
}

variable "initial_release" {
  description = "Release ID (git SHA) instances deploy on first boot; CI updates the SSM parameter afterwards"
  type        = string
}

```

terraform.tfvars.example — Copy to terraform.tfvars (git-ignored).**terraform.tfvars.example** infrastructure/aws/ec2/terraform.tfvars.example

```

# terraform.tfvars.example - copy to terraform.tfvars (git-ignored) and edit.
# Contains NO secrets: passwords live in Secrets Manager, never in tfvars.

project           = "ai-lab"
environment       = "prod"
aws_region        = "us-east-1"
domain_name       = "example.com"           # REPLACE: your Route 53 hosted zone
github_repo       = "OWNER/ai-engineering-lab" # REPLACE
alert_email       = "oncall@example.com"     # REPLACE or leave ""
single_nat_gateway = true

db_instance_class = "db.t4g.medium" # illustrative
db_multi_az       = true

instance_type     = "t3.medium" # illustrative
asg_min_size      = 2
asg_desired_capacity = 2
asg_max_size      = 10
initial_release    = "0123abc" # REPLACE: release you uploaded in the bootstrap step

```

iam.tf — Instance role/profile + GitHub EC2 deploy permissions.**iam.tf** infrastructure/aws/ec2/iam.tf

```

# iam.tf - instance role (what code ON the EC2 instance may do) + CI deploy permissions.
# The instance profile is the container that attaches the role to an instance.

data "aws_iam_policy_document" "ec2_assume" {
  statement {
    actions = ["sts:AssumeRole"]
    principals {
      type       = "Service"
      identifiers = ["ec2.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "api_instance" {
  name           = "${local.name}-api-instance-role"
  assume_role_policy = data.aws_iam_policy_document.ec2_assume.json
}

# Session Manager shell + Run Command + patching, without SSH or a bastion.
resource "aws_iam_role_policy_attachment" "ssm_core" {
  role       = aws_iam_role.api_instance.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}

# Lets the CloudWatch agent ship logs and memory/disk metrics.
resource "aws_iam_role_policy_attachment" "cloudwatch_agent" {
  role       = aws_iam_role.api_instance.name
  policy_arn = "arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy"
}

data "aws_iam_policy_document" "api_instance" {
  statement {
    sid      = "ReadReleaseArtifacts"
    actions  = ["s3:GetObject"]
    resources = ["${aws_s3_bucket.artifacts.arn}/api/*"]
  }
  statement {
    sid      = "ReadReleasePointer"
    actions  = ["ssm:GetParameter"]
    resources = [aws_ssm_parameter.api_release.arn]
  }
  statement {
    sid      = "ReadRuntimeSecrets"
    actions  = ["secretsmanager:GetSecretValue"]
    resources = [local.db_secret_arn, data.aws_secretsmanager_secret.app.arn]
  }
}

resource "aws_iam_role_policy" "api_instance" {
  name     = "api-runtime"
  role     = aws_iam_role.api_instance.id
  policy   = data.aws_iam_policy_document.api_instance.json
}

resource "aws_iam_instance_profile" "api" {
  name = "${local.name}-api-instance-profile"
  role = aws_iam_role.api_instance.name
}

# ---- GitHub Actions: upload artifact, move release pointer, refresh instances ----
data "aws_iam_policy_document" "github_ec2" {
  statement {
    sid      = "UploadArtifacts"
    actions  = ["s3:PutObject", "s3:GetObject"]
    resources = ["${aws_s3_bucket.artifacts.arn}/api/*"]
  }
  statement {
    sid      = "MoveReleasePointer"
    actions  = ["ssm:GetParameter", "ssm:PutParameter"]
    resources = [aws_ssm_parameter.api_release.arn]
  }
  statement {
    sid      = "StartRefresh"
    actions  = ["autoscaling:StartInstanceRefresh", "autoscaling:CancelInstanceRefresh"]
    resources = [aws_autoscaling_group.api.arn]
  }
  statement {
    sid      = "DescribeRefresh"
    actions  = ["autoscaling:DescribeInstanceRefreshes", "autoscaling:DescribeAutoScalingGroups"]
    resources = ["*"] # Describe* actions do not support resource-level permissions
  }
}

resource "aws_iam_role_policy" "github_ec2" {
  name     = "ec2-deploy"
  role     = aws_iam_role.github_deploy.id
  policy   = data.aws_iam_policy_document.github_ec2.json
}

```

ec2.tf — Artifacts, release pointer, logs, launch template, ASG, scaling.

```

ec2.tf infrastructure/aws/ec2/ec2.tf

# ec2.tf - artifact bucket, release pointer, launch template, Auto Scaling group,
# scaling policies and the instance log group.
#
# Release model (immutable instances):
# CI uploads s3://<artifacts>/api/<sha>/app.jar -> sets SSM /ai-lab/prod/api/release = <sha>
# -> starts an ASG instance refresh -> each NEW instance runs user-data, reads the
# parameter, downloads that jar, starts systemd. Old instances are terminated after the
# new ones pass ALB health checks. Roll back = point the parameter at the old SHA + refresh.

locals {
    lb_target_type = "instance"
}

# ---- Artifacts -----
resource "aws_s3_bucket" "artifacts" {
    bucket = "${local.name}-artifacts-${local.account_id}"
}

resource "aws_s3_bucket_public_access_block" "artifacts" {
    bucket             = aws_s3_bucket.artifacts.id
    block_public_acls  = true
    block_public_policy = true
    ignore_public_acls = true
    restrict_public_buckets = true
}

resource "aws_s3_bucket_versioning" "artifacts" {
    bucket = aws_s3_bucket.artifacts.id
    versioning_configuration {
        status = "Enabled"
    }
}

resource "aws_s3_bucket_lifecycle_configuration" "artifacts" {
    bucket = aws_s3_bucket.artifacts.id
    rule {
        id     = "expire-old-releases"
        status = "Enabled"
        filter {
            prefix = "api/"
        }
        expiration {
            days = 90 # keep a 90-day rollback window
        }
        noncurrent_version_expiration {
            noncurrent_days = 30
        }
    }
}

# Pointer to the release that NEW instances install. Terraform creates it once; CI owns
# the value afterwards (ignore_changes stops Terraform from reverting deployments).
resource "aws_ssm_parameter" "api_release" {
    name = "${var.project}/${var.environment}/api/release" # /ai-lab/prod/api/release
    type = "String"
    value = var.initial_release
    lifecycle {
        ignore_changes = [value]
    }
}

# ---- Logs -----
resource "aws_cloudwatch_log_group" "api" {
    name           = "/ec2/${local.name}-api"
    retention_in_days = 30
}

# ---- Launch template -----
# Latest Amazon Linux 2023 AMI (published by AWS as a public SSM parameter). A new AMI
# appears in the plan as a launch-template change -> Terraform triggers an instance refresh.
data "aws_ssm_parameter" "al2023_ami" {
    name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

resource "aws_launch_template" "api" {
    name_prefix      = "${local.name}-api-"
    image_id         = data.aws_ssm_parameter.al2023_ami.insecure_value
    instance_type    = var.instance_type
    vpc_security_group_ids = [aws_security_group.app.id]
    update_default_version = true

    iam_instance_profile {
        arn = aws_iam_instance_profile.api.arn
    }

    # IMDSv2 only; hop limit 1 keeps instance credentials away from containers/proxies.
    metadata_options {
        http_endpoint      = "enabled"
        http_tokens        = "required"
        http_put_response_hop_limit = 1
    }

    block_device_mappings {

```

```

ec2.tf infrastructure/aws/ec2/ec2.tf
    device_name = "/dev/xvda"
    ebs {
        volume_size      = 20
        volume_type       = "gp3"
        encrypted         = true
        delete_on_termination = true
    }
}

monitoring {
    enabled = true # 1-minute EC2 metrics
}

user_data = base64encode(templatefile("${path.module}/templates/user-data.sh.tftpl", {
    aws_region      = var.aws_region
    artifact_bucket = aws_s3_bucket.artifacts.bucket
    release_param   = aws_ssm_parameter.api_release.name
    db_secret_arn   = local.db_secret_arn
    app_secret_arn  = data.aws_secretsmanager_secret.app.arn
    db_host         = aws_db_instance.main.address
    db_name         = var.db_name
    api_port        = var.api_port
    log_group       = aws_cloudwatch_log_group.api.name
    systemd_unit    = file("${path.module}/templates/ai-lab-api.service")
}))

tag_specifications {
    resource_type = "instance"
    tags          = { App = "${local.name}-api" }
}
}

# ---- Auto Scaling group -----
resource "aws_autoscaling_group" "api" {
    name                = "${local.name}-api-asg"
    vpc_zone_identifier = aws_subnet.private[*].id # spread across AZs
    min_size            = var.asg_min_size
    max_size            = var.asg_max_size
    desired_capacity    = var.asg_desired_capacity
    target_group_arns   = [aws_lb_target_group.api.arn]

    health_check_type      = "ELB" # replace instances the ALB marks unhealthy
    health_check_grace_period = 300 # boot + package install + JVM start
    default_instance_warmup = 300

    launch_template {
        id      = aws_launch_template.api.id
        version = aws_launch_template.api.latest_version
    }

    # Terraform starts this refresh automatically when the launch template changes.
    instance_refresh {
        strategy = "Rolling"
        preferences {
            min_healthy_percentage = 100 # launch new before terminating old
            max_healthy_percentage = 200
            instance_warmup        = 300
            auto_rollback          = true # revert to the previous launch template on failure
        }
    }

    enabled_metrics = ["GroupDesiredCapacity", "GroupInServiceInstances", "GroupTotalInstances"]

    tag {
        key      = "Name"
        value    = "${local.name}-api"
        propagate_at_launch = true
    }

    lifecycle {
        ignore_changes = [desired_capacity] # scaling policies own it after creation
    }
}

# ---- Scaling policies (ILLUSTRATIVE targets) -----
resource "aws_autoscaling_policy" "api_cpu" {
    name                = "${local.name}-api-cpu"
    autoscaling_group_name = aws_autoscaling_group.api.name
    policy_type         = "TargetTrackingScaling"
    target_tracking_configuration {
        predefined_metric_specification {
            predefined_metric_type = "ASGAverageCPUUtilization"
        }
    }
    target_value = 50
}

resource "aws_autoscaling_policy" "api_requests" {
    name                = "${local.name}-api-requests"
    autoscaling_group_name = aws_autoscaling_group.api.name
    policy_type         = "TargetTrackingScaling"
    target_tracking_configuration {
        predefined_metric_specification {
            predefined_metric_type = "ALBRequestCountPerTarget"
            resource_label        = "${aws_lb.main.arn_suffix}/${aws_lb_target_group.api.arn_suffix}"
        }
    }
}

```

```
ec2.tf infrastructure/aws/ec2/ec2.tf
}
target_value = 1000 # requests per target per minute - REPLACE with a load-tested value
}
}
```

templates/user-data.sh.tftpl — First-boot bootstrap (Terraform template).**templates/user-data.sh.tftpl** infrastructure/aws/ec2/templates/user-data.sh.tftpl

```
#!/bin/bash
# user-data.sh.tftpl - rendered by Terraform templatefile() into the launch template.
# cloud-init runs it ONCE, as root, on the first boot of every new instance.
# Terraform fills every dollar-brace placeholder; a doubled dollar sign ($$) escapes a literal one.
set -euo pipefail
exec >>(tee -a /var/log/user-data.log) 2>&1
echo "user-data start: $(date -Is)"

# 1. OS packages (Amazon Linux 2023 already ships the AWS CLI v2 and the SSM agent).
dnf install -y java-21-amazon-corretto-headless amazon-cloudwatch-agent jq

# 2. Service account and directories.
id ailaab >/dev/null 2>&1 || useradd --system --home-dir /opt/ai-lab --shell /sbin/nologin ailaab
install -d -o ailaab -g ailaab -m 0750 /opt/ai-lab /opt/ai-lab/releases
install -d -o ailaab -g ailaab -m 0750 /var/log/ai-lab
install -d -m 0755 /etc/ai-lab

# 3. Deployment settings (non-secret) used by the helper scripts.
cat > /etc/ai-lab/deploy.env <<CONF
AWS_REGION=${aws_region}
ARTIFACT_BUCKET=${artifact_bucket}
RELEASE_PARAM=${release_param}
DB_SECRET_ARN=${db_secret_arn}
APP_SECRET_ARN=${app_secret_arn}
CONF

# 4. Application runtime configuration (non-secret).
cat > /etc/ai-lab/api.env <<CONF
SERVER_PORT=${api_port}
POSTGRES_HOST=${db_host}
POSTGRES_PORT=5432
POSTGRES_DB=${db_name}
CONF
chmod 0644 /etc/ai-lab/*.env

# 5. Secret fetcher: runs before every service start, writes a root-only tmpfs file.
cat > /usr/local/bin/ai-lab-fetch-secrets <<'SCRIPT'
#!/bin/bash
set -euo pipefail
source /etc/ai-lab/deploy.env
install -d -m 0700 /run/ai-lab
umask 077
db_json=$(aws secretsmanager get-secret-value --region "$AWS_REGION" \
--secret-id "$DB_SECRET_ARN" --query SecretString --output text)
app_json=$(aws secretsmanager get-secret-value --region "$AWS_REGION" \
--secret-id "$APP_SECRET_ARN" --query SecretString --output text)
{
    echo "POSTGRES_USER=$(jq -r .username <<<"$db_json")"
    echo "POSTGRES_PASSWORD=$(jq -r .password <<<"$db_json")"
    jq -r 'to_entries[] | "\(.key)=\(.value)"' <<<"$app_json"
} > /run/ai-lab/secrets.env.tmp
mv /run/ai-lab/secrets.env.tmp /run/ai-lab/secrets.env
SCRIPT

# 6. Release installer: downloads the jar named by the SSM release pointer.
cat > /usr/local/bin/ai-lab-deploy <<'SCRIPT'
#!/bin/bash
set -euo pipefail
source /etc/ai-lab/deploy.env
release=$(aws ssm get-parameter --region "$AWS_REGION" --name "$RELEASE_PARAM" \
--query Parameter.Value --output text)
dest="/opt/ai-lab/releases/$release"
install -d -o ailaab -g ailaab -m 0750 "$dest"
aws s3 cp --region "$AWS_REGION" "s3://$ARTIFACT_BUCKET/api/$release/app.jar" "$dest/app.jar"
chown ailaab:ailaab "$dest/app.jar"
ln -sf "$dest" /opt/ai-lab/current
systemctl daemon-reload
systemctl enable ai-lab-api
systemctl restart ai-lab-api
echo "deployed release $release"
SCRIPT
chmod 0750 /usr/local/bin/ai-lab-fetch-secrets /usr/local/bin/ai-lab-deploy

# 7. systemd unit (content comes from templates/ai-lab-api.service).
cat > /etc/systemd/system/ai-lab-api.service <<'UNIT'
$[systemd_unit]
UNIT

# 8. Log rotation for the file the unit appends to.
cat > /etc/logrotate.d/ai-lab <<'ROTATE'
/var/log/ai-lab/*.log {
    daily
    rotate 7
    compress
    missingok
    copytruncate
}
ROTATE

# 9. CloudWatch agent: ship app + bootstrap logs, collect memory and disk metrics.
cat > /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json <<'CWA'
{
    "agent": { "metrics_collection_interval": 60, "run_as_user": "root" },
```

templates/user-data.sh.tftpl infrastructure/aws/ec2/templates/user-data.sh.tftpl

```

"logs": {
  "logs_collected": {
    "files": {
      "collect_list": [
        { "file_path": "/var/log/ai-lab/api.log",
          "log_group_name": "${log_group}", "log_stream_name": "{instance_id}/app" },
        { "file_path": "/var/log/user-data.log",
          "log_group_name": "${log_group}", "log_stream_name": "{instance_id}/user-data" }
      ]
    }
  },
  "metrics": {
    "namespace": "CWAgent",
    "append_dimensions": {
      "AutoScalingGroupName": "${aws:AutoScalingGroupName}",
      "InstanceId": "${aws:InstanceId}"
    },
    "aggregation_dimensions": [ ["AutoScalingGroupName"] ],
    "metrics_collected": {
      "mem": { "measurement": [ "mem_used_percent" ] },
      "disk": { "measurement": [ "used_percent" ], "resources": [ "/" ] }
    }
  }
}
CWA
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -s \
-c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json

# 10. Install and start the current release. The ALB health check decides when the
# instance receives traffic; the ASG replaces it if it never becomes healthy.
/usr/local/bin/ai-lab-deploy
echo "user-data done: $(date -Is)"

```

templates/ai-lab-api.service — systemd unit embedded by user-data.**templates/ai-lab-api.service** infrastructure/aws/ec2/templates/ai-lab-api.service

```

# /etc/systemd/system/ai-lab-api.service - keeps the API process running on the instance.
# Installed by user-data. Secrets are fetched at every (re)start into a tmpfs file.
[Unit]
Description=ai-lab API
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=ailab
Group=ailab
WorkingDirectory=/opt/ai-lab/current

# Non-secret config (written once by user-data) + secrets (written by ExecStartPre).
EnvironmentFile=/etc/ai-lab/api.env
EnvironmentFile=-/run/ai-lab/secrets.env
# "+" runs this one command as root so it can write /run/ai-lab with mode 0600.
ExecStartPre+=/usr/local/bin/ai-lab-fetch-secrets

ExecStart=/usr/bin/java -XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError -jar /opt/ai-lab/current/app.jar
# JVM exits with 143 after SIGTERM: treat as a clean stop.
SuccessExitStatus=143
Restart=on-failure
RestartSec=5
TimeoutStopSec=30

StandardOutput=append:/var/log/ai-lab/api.log
StandardError=append:/var/log/ai-lab/api.log

# Hardening
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/opt/ai-lab
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target

```

scripts/deploy-release.sh — Deploy/rollback via instance refresh.**scripts/deploy-release.sh** infrastructure/aws/ec2/scripts/deploy-release.sh

```
#!/usr/bin/env bash
# deploy-release.sh - release (or roll back) the API on the EC2 Auto Scaling group.
# Used by GitHub Actions and by operators. Requires AWS credentials in the environment.
#
#   Deploy a new build : scripts/deploy-release.sh <release-id> path/to/app.jar
#   Roll back          : scripts/deploy-release.sh <previous-release-id>
#                       (jar already in S3, so no upload)
set -euo pipefail

release="${1:?usage: deploy-release.sh <release-id> [app.jar]}"
jar="${2:-}"

: "${AWS_REGION:?set AWS_REGION}"
: "${ARTIFACT_BUCKET:?set ARTIFACT_BUCKET}" # ai-lab-prod-artifacts-<account-id>
: "${ASG_NAME:?set ASG_NAME}" # ai-lab-prod-api-asg
: "${RELEASE_PARAM:?set RELEASE_PARAM}" # /ai-lab/prod/api/release

key="api/${release}/app.jar"

if [[ -n "$jar" ]]; then
    echo "Uploading $jar to s3://${ARTIFACT_BUCKET}/${key}"
    aws s3 cp "$jar" "s3://${ARTIFACT_BUCKET}/${key}" --region "$AWS_REGION"
else
    echo "Checking that s3://${ARTIFACT_BUCKET}/${key} exists"
    aws s3api head-object --bucket "$ARTIFACT_BUCKET" --key "$key" --region "$AWS_REGION" >/dev/null
fi

previous=$(aws ssm get-parameter --name "$RELEASE_PARAM" --region "$AWS_REGION" \
    --query Parameter.Value --output text)
echo "Release pointer: $previous -> $release"
aws ssm put-parameter --name "$RELEASE_PARAM" --value "$release" --type String \
    --overwrite --region "$AWS_REGION" >/dev/null

refresh_id=$(aws autoscaling start-instance-refresh \
    --auto-scaling-group-name "$ASG_NAME" \
    --strategy Rolling \
    --preferences '{"MinHealthyPercentage":100,"MaxHealthyPercentage":200,"InstanceWarmup":300}' \
    --region "$AWS_REGION" --query InstanceRefreshId --output text)
echo "Instance refresh $refresh_id started"

while true; do
    read -r status pct < <(aws autoscaling describe-instance-refreshes \
        --auto-scaling-group-name "$ASG_NAME" --instance-refresh-ids "$refresh_id" \
        --region "$AWS_REGION" \
        --query 'InstanceRefreshes[0].[Status,PercentageComplete]' --output text)
    echo "${date -u +%H:%M:%S} status=$status complete=${pct}%"
    case "$status" in
        Successful) echo "Release $release is live"; exit 0 ;;
        Failed|Cancelled|RollbackSuccessful|RollbackFailed)
            echo "Refresh ended with $status. Restoring pointer to $previous" >&2
            aws ssm put-parameter --name "$RELEASE_PARAM" --value "$previous" --type String \
                --overwrite --region "$AWS_REGION" >/dev/null
            exit 1 ;;
    esac
    sleep 30
done
```

outputs.tf — Values for CI and operators.

outputs.tf infrastructure/aws/ec2/outputs.tf

```
# outputs.tf - values the CI workflow and operators need after apply.
# Read with: terraform output -raw <name>

output "app_url" {
  value = "https://${local.app_fqdn}"
}

output "api_url" {
  value = "https://${local.api_fqdn}"
}

output "alb_dns_name" {
  value = aws_lb.main.dns_name
}

output "asg_name" {
  value = aws_autoscaling_group.api.name
}

output "artifact_bucket" {
  value = aws_s3_bucket.artifacts.bucket
}

output "release_parameter" {
  value = aws_ssm_parameter.api_release.name
}

output "frontend_bucket" {
  value = aws_s3_bucket.frontend.bucket
}

output "cloudfront_distribution_id" {
  value = aws_cloudfront_distribution.frontend.id
}

output "db_endpoint" {
  value = aws_db_instance.main.address
}

output "db_secret_arn" {
  value = local.db_secret_arn
}

output "github_deploy_role_arn" {
  value = aws_iam_role.github_deploy.arn
}
```

C · ECS Fargate stack (infrastructure/aws/ecs/)

versions.tf — Pins + S3 backend (key prod/ecs/...).

versions.tf infrastructure/aws/ecs/versions.tf

```
# versions.tf - pins Terraform + provider versions and stores state remotely.
# The state bucket is created ONCE by hand (see the bootstrap step); never commit state.

terraform {
  required_version = ">= 1.10.0" # S3 native state locking (use_lockfile)

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 6.0"
    }
  }

  backend "s3" {
    bucket = "ai-lab-tfstate-111122223333" # REPLACE: your state bucket (account ID suffix)
    key    = "prod/ecs/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    use_lockfile = true # lock file in S3; no DynamoDB table needed
  }
}
```

variables.tf — All inputs incl. task size, counts, initial_image_tag.**variables.tf** infrastructure/aws/ecs/variables.tf

variables.tf - every input of the ECS Fargate stack. Values go in terraform.tfvars.

```

variable "project" {
  description = "Short project name used as a prefix for every resource"
  type        = string
  default     = "ai-lab"
}

variable "environment" {
  description = "Environment name (prod, staging, ...)"
  type        = string
  default     = "prod"
}

variable "aws_region" {
  description = "Region for the VPC, ALB, compute and RDS"
  type        = string
  default     = "us-east-1"
}

variable "domain_name" {
  description = "Apex domain of an existing Route 53 public hosted zone"
  type        = string
}

variable "app_subdomain" {
  description = "Subdomain for the React app (CloudFront)"
  type        = string
  default     = "app"
}

variable "api_subdomain" {
  description = "Subdomain for the backend API (ALB)"
  type        = string
  default     = "api"
}

variable "github_repo" {
  description = "GitHub repository allowed to deploy, as OWNER/REPO"
  type        = string
}

variable "alert_email" {
  description = "Email for CloudWatch alarm notifications (empty = none)"
  type        = string
  default     = ""
}

# --- Network -----
variable "vpc_cidr" {
  type        = string
  default     = "10.20.0.0/16"
}

variable "az_count" {
  description = "Number of Availability Zones (2 minimum for ALB and Multi-AZ RDS)"
  type        = number
  default     = 2
  validation {
    condition   = var.az_count >= 2 && var.az_count <= 3
    error_message = "az_count must be 2 or 3."
  }
}

variable "single_nat_gateway" {
  description = "true = one NAT (cheaper, AZ-level single point of failure); false = one per AZ"
  type        = bool
  default     = true
}

# --- API -----
variable "api_port" {
  type        = number
  default     = 8080
}

variable "health_check_path" {
  type        = string
  default     = "/actuator/health"
}

# --- Database -----
variable "db_engine_version" {
  type        = string
  default     = "17"
}

variable "db_instance_class" {
  type        = string
  default     = "db.t4g.medium"
}

variable "db_allocated_storage" {

```

variables.tf infrastructure/aws/ecs/variables.tf

```

type      = number
default   = 50
}

variable "db_max_allocated_storage" {
  type      = number
  default   = 200
}

variable "db_multi_az" {
  type      = bool
  default   = true
}

variable "db_name" {
  type      = string
  default   = "ai_lab"
}

variable "db_username" {
  type      = string
  default   = "ai_lab_admin"
}

variable "db_connections_alarm_threshold" {
  description = "Alarm when connections exceed this; derive from SHOW max_connections"
  type        = number
  default     = 300
}

# --- ECS Fargate (ILLUSTRATIVE sizes: confirm with load tests) -----
variable "task_cpu" {
  description = "Task CPU units (1024 = 1 vCPU)"
  type        = number
  default     = 1024
}

variable "task_memory" {
  description = "Task memory in MiB (must be a valid Fargate pairing with task_cpu)"
  type        = number
  default     = 2048
}

variable "initial_image_tag" {
  description = "Image tag used only when Terraform first creates the task definition"
  type        = string
}

variable "service_desired_count" {
  type      = number
  default   = 2
}

variable "service_min_count" {
  type      = number
  default   = 2
}

variable "service_max_count" {
  type      = number
  default   = 10
}

```

terraform.tfvars.example — Copy to terraform.tfvars (git-ignored).**terraform.tfvars.example** infrastructure/aws/ecs/terraform.tfvars.example

```

# terraform.tfvars.example - copy to terraform.tfvars (git-ignored) and edit.
# Contains NO secrets: passwords live in Secrets Manager, never in tfvars.

project      = "ai-lab"
environment  = "prod"
aws_region   = "us-east-1"
domain_name  = "example.com" # REPLACE: your Route 53 hosted zone
github_repo  = "OWNER/ai-engineering-lab" # REPLACE
alert_email  = "oncall@example.com" # REPLACE or leave ""
single_nat_gateway = true

db_instance_class = "db.t4g.medium" # illustrative
db_multi_az       = true

task_cpu          = 1024 # illustrative
task_memory       = 2048 # illustrative
initial_image_tag = "0123abc" # REPLACE: tag you pushed in the bootstrap step
service_desired_count = 2
service_min_count  = 2
service_max_count  = 10

```

iam.tf — Execution role, task role, GitHub ECS deploy permissions.**iam.tf** infrastructure/aws/ecs/iam.tf

```

# iam.tf - the three ECS-related roles plus the CI deploy permissions.
# execution role : used by the ECS AGENT before your code runs (pull image, read
#                 secrets, create log stream). Your application never sees it.
# task role      : used by YOUR APPLICATION at runtime (AWS SDK calls). Least privilege.
# github deploy  : used by GitHub Actions through OIDC (defined in github-oidc.tf).

data "aws_iam_policy_document" "ecs_tasks_assume" {
  statement {
    actions = ["sts:AssumeRole"]
    principals {
      type        = "Service"
      identifiers = ["ecs-tasks.amazonaws.com"]
    }
    condition {
      test     = "StringEquals"
      variable = "aws:SourceAccount"
      values   = [local.account_id]
    }
  }
}

# ---- Task execution role -----
resource "aws_iam_role" "ecs_execution" {
  name               = "${local.name}-api-execution-role"
  assume_role_policy = data.aws_iam_policy_document.ecs_tasks_assume.json
}

resource "aws_iam_role_policy_attachment" "ecs_execution_managed" {
  role       = aws_iam_role.ecs_execution.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy"
}

data "aws_iam_policy_document" "ecs_execution_secrets" {
  statement {
    sid = "ReadInjectedSecrets"
    actions = ["secretsmanager:GetSecretValue"]
    resources = [local.db_secret_arn, data.aws_secretsmanager_secret.app.arn]
  }
  # Add kms:Decrypt on the key ARN only if a secret uses a customer-managed KMS key.
}

resource "aws_iam_role_policy" "ecs_execution_secrets" {
  name   = "read-secrets"
  role   = aws_iam_role.ecs_execution.id
  policy = data.aws_iam_policy_document.ecs_execution_secrets.json
}

# ---- Task role (application) -----
resource "aws_iam_role" "ecs_task" {
  name               = "${local.name}-api-task-role"
  assume_role_policy = data.aws_iam_policy_document.ecs_tasks_assume.json
}

# OPTIONAL: lets operators open a shell with ECS Exec (enable_execute_command = true).
data "aws_iam_policy_document" "ecs_task_exec" {
  statement {
    sid = "EcsExecChannels"
    actions = [
      "ssmmessages:CreateControlChannel",
      "ssmmessages:CreateDataChannel",
      "ssmmessages:OpenControlChannel",
      "ssmmessages:OpenDataChannel",
    ]
    resources = ["*"]
  }
}

resource "aws_iam_role_policy" "ecs_task_exec" {
  name   = "ecs-exec"
  role   = aws_iam_role.ecs_task.id
  policy = data.aws_iam_policy_document.ecs_task_exec.json
}

# ---- GitHub Actions: push image + roll the service -----
data "aws_iam_policy_document" "github_ecs" {
  statement {
    sid = "EcrLogin"
    actions = ["ecr:GetAuthorizationToken"]
    resources = ["*"] # this action does not support resource-level permissions
  }
  statement {
    sid = "EcrPushPull"
    actions = [
      "ecr:BatchCheckLayerAvailability",
      "ecr:BatchGetImage",
      "ecr:CompleteLayerUpload",
      "ecr:DescribeImages",
      "ecr:InitiateLayerUpload",
      "ecr:PutImage",
      "ecr:UploadLayerPart",
    ]
    resources = [aws_ecr_repository.api.arn]
  }
}

```

```
iam.tf infrastructure/aws/ecs/iam.tf

statement {
  sid      = "TaskDefinitions"
  actions  = ["ecs:DescribeTaskDefinition", "ecs:RegisterTaskDefinition"]
  resources = ["*"] # these actions do not support resource-level permissions
}
statement {
  sid      = "RollService"
  actions  = ["ecs:UpdateService", "ecs:DescribeServices"]
  resources = [aws_ecs_service.api.id] # the service ARN
}
statement {
  sid      = "PassOnlyTheTaskRoles"
  actions  = ["iam:PassRole"]
  resources = [aws_iam_role.ecs_execution.arn, aws_iam_role.ecs_task.arn]
  condition {
    test     = "StringEquals"
    variable = "iam:PassedToService"
    values   = ["ecs-tasks.amazonaws.com"]
  }
}

resource "aws_iam_role_policy" "github_ecs" {
  name     = "ecs-deploy"
  role     = aws_iam_role.github_deploy.id
  policy   = data.aws_iam_policy_document.github_ecs.json
}
```

ecr.tf — Repository + lifecycle policy.

```
ecr.tf infrastructure/aws/ecs/ecr.tf

# ecr.tf - private registry for the API image. Tags are IMMUTABLE: tag = git SHA, so a
# tag always means exactly one build and rollbacks are deterministic.

resource "aws_ecr_repository" "api" {
  name                = "${var.project}/api" # ai-lab/api
  image_tag_mutability = "IMMUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }

  encryption_configuration {
    encryption_type = "AES256"
  }
}

resource "aws_ecr_lifecycle_policy" "api" {
  repository = aws_ecr_repository.api.name
  policy     = jsonencode({
    rules = [{
      rulePriority = 1
      description = "Keep the 30 most recent images (rollback window)"
      selection = {
        tagStatus    = "any"
        countType    = "imageCountMoreThan"
        countNumber = 30
      }
      action = { type = "expire" }
    }]
  })
}
```

ecs-cluster.tf — Cluster + log group.

```
ecs-cluster.tf infrastructure/aws/ecs/ecs-cluster.tf

# ecs-cluster.tf - the logical cluster and the log group the containers write to.
# With Fargate there are no instances to register: the cluster is just a namespace.

resource "aws_ecs_cluster" "main" {
  name = local.name # ai-lab-prod

  setting {
    name = "containerInsights"
    value = "enabled" # per-task CPU/memory metrics (extra CloudWatch cost)
  }
}

resource "aws_cloudwatch_log_group" "api" {
  name                = "/ecs/${local.name}-api"
  retention_in_days = 30
}
```

ecs-task-definition.tf — Task definition (Terraform form).**ecs-task-definition.tf** infrastructure/aws/ecs/ecs-task-definition.tf

```
# ecs-task-definition.tf - WHAT to run: image, CPU/memory, ports, env, secrets, health
# check, logs. Each change creates a new immutable REVISION (ai-lab-prod-api:1, :2, ...).
# Terraform owns the SHAPE; CI only swaps the image and registers new revisions.

resource "aws_ecs_task_definition" "api" {
  family             = "${local.name}-api"
  requires_compatibilities = ["FARGATE"]
  network_mode       = "awsvpc"
  cpu                = var.task_cpu
  memory             = var.task_memory
  execution_role_arn = aws_iam_role.ecs_execution.arn
  task_role_arn      = aws_iam_role.ecs_task.arn

  runtime_platform {
    operating_system_family = "LINUX"
    cpu_architecture        = "X86_64" # use ARM64 only if the image is built for arm64
  }

  container_definitions = jsonencode([
    {
      name       = "api"
      image      = "${aws_ecr_repository.api.repository_url}:${var.initial_image_tag}"
      essential = true

      portMappings = [{ containerPort = var.api_port, protocol = "tcp" }]

      # Plain, non-sensitive configuration.
      environment = [
        { name = "SERVER_PORT", value = tostring(var.api_port) },
        { name = "POSTGRES_HOST", value = aws_db_instance.main.address },
        { name = "POSTGRES_PORT", value = "5432" },
        { name = "POSTGRES_DB", value = var.db_name },
      ]

      # Resolved by the ECS agent at task start using the EXECUTION role.
      # Format: <secret-arn>:<json-key>:<version-stage>:<version-id>
      secrets = [
        { name = "POSTGRES_USER", valueFrom = "${local.db_secret_arn}:username:" },
        { name = "POSTGRES_PASSWORD", valueFrom = "${local.db_secret_arn}:password:" },
        { name = "OPENAI_API_KEY", valueFrom = "${data.aws_secretsmanager_secret.app.arn}:OPENAI_API_KEY:" },
      ]

      # Container-level health check (runs inside the container; image needs curl).
      healthCheck = {
        command = ["CMD-SHELL", "curl -fsS http://localhost:${var.api_port}${var.health_check_path} || exit 1"]
        interval = 30
        timeout  = 5
        retries  = 3
        startPeriod = 120 # JVM start-up grace; failures are ignored during this window
      }

      logConfiguration = {
        logDriver = "awslogs"
        options = {
          awslogs-group      = aws_cloudwatch_log_group.api.name
          awslogs-region     = var.aws_region
          awslogs-stream-prefix = "api"
        }
      }

      stopTimeout = 30 # seconds between SIGTERM and SIGKILL (graceful shutdown)
    }
  ])
}
```

ecs-service.tf — Service, rollout, circuit breaker, alarm rollback.**ecs-service.tf** infrastructure/aws/ecs/ecs-service.tf

```
# ecs-service.tf - HOW MANY copies run, WHERE (private subnets, 2 AZs), behind WHICH
# target group, and HOW deployments roll (rolling + circuit breaker + alarm rollback).

locals {
  lb_target_type = "ip" # Fargate tasks register by ENI IP
}

resource "aws_ecs_service" "api" {
  name           = "${local.name}-api"
  cluster        = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.api.arn
  desired_count  = var.service_desired_count
  launch_type    = "FARGATE"
  platform_version = "LATEST"
  enable_execute_command = true # OPTIONAL: ECS Exec for debugging
  propagate_tags   = "SERVICE"

  # Ignore ALB health checks for this long after a task starts (JVM warm-up).
  health_check_grace_period_seconds = 120

  # Rolling update: start new tasks first (up to 200%), never drop below 100% healthy.
  deployment_minimum_healthy_percent = 100
  deployment_maximum_percent         = 200

  deployment_controller {
    type = "ECS"
  }

  # Roll back automatically when new tasks keep failing to start or to pass health checks.
  deployment_circuit_breaker {
    enable   = true
    rollback = true
  }

  # Also roll back when this alarm fires during the deployment.
  alarms {
    enable   = true
    rollback = true
    alarm_names = [aws_cloudwatch_metric_alarm.api_5xx.alarm_name]
  }

  network_configuration {
    subnets          = aws_subnet.private[*].id
    security_groups    = [aws_security_group.app.id]
    assign_public_ip   = false
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.api.arn
    container_name   = "api"
    container_port    = var.api_port
  }

  depends_on = [aws_lb_listener.https]

  lifecycle {
    # task_definition: CI registers new revisions and points the service at them.
    # desired_count   : Service Auto Scaling owns it after creation.
    ignore_changes = [task_definition, desired_count]
  }
}
```

autoscaling.tf — Scalable target + target tracking.**autoscaling.tf** infrastructure/aws/ecs/autoscaling.tf

```
# autoscaling.tf - Service Auto Scaling adjusts desired_count between min and max.
# Two target-tracking policies: whichever asks for MORE tasks wins; scale-in happens only
# when both agree. All numbers are ILLUSTRATIVE until confirmed by load testing.

resource "aws_appautoscaling_target" "api" {
  service_namespace = "ecs"
  scalable_dimension = "ecs:service:DesiredCount"
  resource_id        = "service/${aws_ecs_cluster.main.name}/${aws_ecs_service.api.name}"
  min_capacity       = var.service_min_count
  max_capacity       = var.service_max_count
}

resource "aws_appautoscaling_policy" "api_cpu" {
  name           = "${local.name}-api-cpu"
  policy_type    = "TargetTrackingScaling"
  service_namespace = aws_appautoscaling_target.api.service_namespace
  resource_id     = aws_appautoscaling_target.api.resource_id
  scalable_dimension = aws_appautoscaling_target.api.scalable_dimension

  target_tracking_scaling_policy_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ECSServiceAverageCPUUtilization"
    }
    target_value      = 50 # keep average CPU near 50% to leave burst headroom
    scale_out_cooldown = 60 # seconds
    scale_in_cooldown = 300 # scale in slowly
  }
}

resource "aws_appautoscaling_policy" "api_requests" {
  name           = "${local.name}-api-requests"
  policy_type    = "TargetTrackingScaling"
  service_namespace = aws_appautoscaling_target.api.service_namespace
  resource_id     = aws_appautoscaling_target.api.resource_id
  scalable_dimension = aws_appautoscaling_target.api.scalable_dimension

  target_tracking_scaling_policy_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ALBRequestCountPerTarget"
      resource_label        = "${aws_lb.main.arn_suffix}/${aws_lb_target_group.api.arn_suffix}"
    }
    target_value      = 1000 # requests per target per minute - REPLACE with a load-tested value
    scale_out_cooldown = 60
    scale_in_cooldown = 300
  }
}
```

outputs.tf — Values for CI and operators.

```
outputs.tf infrastructure/aws/ecs/outputs.tf
```

```
# outputs.tf - values the CI workflow and operators need after apply.
```

```
# Read with: terraform output -raw <name>
```

```
output "app_url" {  
  value = "https://${local.app_fqdn}"  
}
```

```
output "api_url" {  
  value = "https://${local.api_fqdn}"  
}
```

```
output "alb_dns_name" {  
  value = aws_lb.main.dns_name  
}
```

```
output "ecr_repository_url" {  
  value = aws_ecr_repository.api.repository_url  
}
```

```
output "ecs_cluster_name" {  
  value = aws_ecs_cluster.main.name  
}
```

```
output "ecs_service_name" {  
  value = aws_ecs_service.api.name  
}
```

```
output "task_definition_family" {  
  value = aws_ecs_task_definition.api.family  
}
```

```
output "frontend_bucket" {  
  value = aws_s3_bucket.frontend.bucket  
}
```

```
output "cloudfront_distribution_id" {  
  value = aws_cloudfront_distribution.frontend.id  
}
```

```
output "db_endpoint" {  
  value = aws_db_instance.main.address  
}
```

```
output "db_secret_arn" {  
  value = local.db_secret_arn  
}
```

```
output "github_deploy_role_arn" {  
  value = aws_iam_role.github_deploy.arn  
}
```

D · Container packaging

services/ai-engineering-api/Dockerfile — Two-stage image build.

services/ai-engineering-api/Dockerfile

```
# syntax=docker/dockerfile:1
# Dockerfile - packages the backend as an OCI image for ECS Fargate.
# Build context: services/ai-engineering-api   Output: <ecr-url>/ai-lab/api:<git-sha>
# Production tip: pin both base images by digest (image@sha256:...) for reproducible builds.

# ---- Stage 1: build the jar (tests already ran in CI before this step) -----
FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN chmod +x mvnw && ./mvnw -B -q dependency:go-offline
COPY src/ src/
RUN ./mvnw -B -q package -DskipTests && cp target/*.jar /workspace/app.jar

# ---- Stage 2: minimal runtime -----
FROM eclipse-temurin:21-jre
# curl is used only by the ECS container health check.
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/* \
    && groupadd --system app \
    && useradd --system --gid app --home-dir /app --create-home app
WORKDIR /app
COPY --from=build --chown=app:app /workspace/app.jar /app/app.jar
USER app
EXPOSE 8080
# Size the heap from the TASK memory limit, not the host; exit (and get replaced) on OOM.
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError"
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

services/ai-engineering-api/.dockerignore — Build-context exclusions.

services/ai-engineering-api/.dockerignore

```
# .dockerignore - keeps the build context small and secrets out of image layers.
target/
.git
.gitignore
.idea/
.vscode/
*.iml
*.log
.env
.env.*
requests.http
Dockerfile
.dockerignore
```

E · GitHub Actions workflows

.github/workflows/deploy-ecs.yml — Approach B pipeline.

```
.github/workflows/deploy-ecs.yml

# .github/workflows/deploy-ecs.yml - test -> build image -> push to ECR -> roll ECS service
# -> publish React to S3 -> invalidate CloudFront. AWS access via OIDC (no stored keys).
name: deploy-ecs

on:
  push:
    branches: [main]
  workflow_dispatch: {} # manual re-deploy, e.g. after a Terraform task-definition change

permissions:
  id-token: write # required to request the GitHub OIDC token
  contents: read

concurrency:
  group: deploy-prod # never run two production deploys at once
  cancel-in-progress: false

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: ai-lab/api
  ECS_CLUSTER: ai-lab-prod
  ECS_SERVICE: ai-lab-prod-api
  TASK_FAMILY: ai-lab-prod-api
  CONTAINER_NAME: api
  API_DIR: services/ai-engineering-api
  WEB_DIR: frontend

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: Unit and integration tests
        working-directory: ${ env.API_DIR }
        run: ./mvnw -B verify

  deploy-api:
    needs: test
    runs-on: ubuntu-latest
    environment: production # must match the IAM trust policy "sub" condition
    timeout-minutes: 40
    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ vars.AWS_DEPLOY_ROLE_ARN } # terraform output github_deploy_role_arn
          aws-region: ${ env.AWS_REGION }

      - name: Log in to Amazon ECR
        id: ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build and push image (tag = git SHA, immutable)
        id: image
        env:
          REGISTRY: ${ steps.ecr.outputs.registry }
          IMAGE_TAG: ${ github.sha }
        run: |
          IMAGE="$REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG"
          if aws ecr describe-images --repository-name "$ECR_REPOSITORY" \
            --image-ids imageTag="$IMAGE_TAG" >/dev/null 2>&1; then
            echo "Image $IMAGE already exists (re-run); skipping build"
          else
            docker build -t "$IMAGE" "$API_DIR"
            docker push "$IMAGE"
          fi
          echo "image=$IMAGE" >> "$GITHUB_OUTPUT"

      - name: Fetch the latest task definition (Terraform owns its shape)
        run: |
          aws ecs describe-task-definition --task-definition "$TASK_FAMILY" \
            --query taskDefinition > taskdef.json

      - name: Swap in the new image
        id: render
        uses: aws-actions/amazon-ecs-render-task-definition@v1
        with:
          task-definition: taskdef.json
          container-name: ${ env.CONTAINER_NAME }
          image: ${ steps.image.outputs.image }

      - name: Register revision and roll the service
        uses: aws-actions/amazon-ecs-deploy-task-definition@v2
        with:
          task-definition: ${ steps.render.outputs.task-definition }
          cluster: ${ env.ECS_CLUSTER }
```

```

.github/workflows/deploy-ecs.yml
  service: ${ env.ECS_SERVICE }}
  wait-for-service-stability: true # fails the job if ECS rolls back

deploy-web:
  needs: deploy-api # ship the UI only after the API it calls is healthy
  runs-on: ubuntu-latest
  environment: production
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: "22"
        cache: npm
        cache-dependency-path: ${ env.WEB_DIR }}/package-lock.json
    - name: Build React (API URL is baked in at build time)
      working-directory: ${ env.WEB_DIR }}
      env:
        VITE_API_BASE_URL: https://api.example.com # REPLACE
      run: |
        npm ci
        npm run build
    - uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${ vars.AWS_DEPLOY_ROLE_ARN }}
        aws-region: ${ env.AWS_REGION }}
    - name: Upload to S3 and invalidate CloudFront
      env:
        BUCKET: ${ vars.FRONTEND_BUCKET }} # terraform output frontend_bucket
        DISTRIBUTION_ID: ${ vars.CLOUDFRONT_DISTRIBUTION_ID }}
      run: |
        # Hashed assets: cache for a year. index.html: always revalidate.
        aws s3 sync "$WEB_DIR/dist" "s3://$BUCKET" --delete \
          --exclude index.html --cache-control "public,max-age=31536000,immutable"
        aws s3 cp "$WEB_DIR/dist/index.html" "s3://$BUCKET/index.html" \
          --cache-control "no-cache"
        aws cloudfront create-invalidation --distribution-id "$DISTRIBUTION_ID" \
          --paths "/index.html"

```

.github/workflows/deploy-ec2.yml — Approach A pipeline (+ rollback input).

```
.github/workflows/deploy-ec2.yml

# .github/workflows/deploy-ec2.yml - test + package jar -> upload to S3 -> move the SSM
# release pointer -> ASG instance refresh -> publish React. AWS access via OIDC.
name: deploy-ec2

on:
  push:
    branches: [main]
  workflow_dispatch:
    inputs:
      release:
        description: "Existing release ID (git SHA) to roll back to. Empty = build HEAD."
        required: false
        default: ""

permissions:
  id-token: write
  contents: read

concurrency:
  group: deploy-prod
  cancel-in-progress: false

env:
  AWS_REGION: us-east-1
  ASG_NAME: ai-lab-prod-api-asg
  RELEASE_PARAM: /ai-lab/prod/api/release
  API_DIR: services/ai-engineering-api
  WEB_DIR: frontend

jobs:
  deploy-api:
    runs-on: ubuntu-latest
    environment: production
    timeout-minutes: 60
    env:
      ARTIFACT_BUCKET: ${vars.ARTIFACT_BUCKET} # terraform output artifact_bucket
      ROLLBACK_TO: ${inputs.release}
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        if: env.ROLLBACK_TO == ''
        with:
          distribution: temurin
          java-version: "21"
          cache: maven

      - name: Test and package
        if: env.ROLLBACK_TO == ''
        working-directory: ${env.API_DIR}
        run: |
          ./mvnw -B verify
          cp target/*.jar "$GITHUB_WORKSPACE/app.jar"

      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${vars.AWS_DEPLOY_ROLE_ARN}
          aws-region: ${env.AWS_REGION}

      - name: Deploy new release
        if: env.ROLLBACK_TO == ''
        run: infrastructure/aws/ec2/scripts/deploy-release.sh "$GITHUB_SHA" app.jar

      - name: Roll back to an existing release
        if: env.ROLLBACK_TO != ''
        run: infrastructure/aws/ec2/scripts/deploy-release.sh "$ROLLBACK_TO"

  deploy-web:
    needs: deploy-api
    if: inputs.release == '' # a backend rollback does not rebuild the UI
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
          cache-dependency-path: ${env.WEB_DIR}/package-lock.json
      - name: Build React
        working-directory: ${env.WEB_DIR}
        env:
          VITE_API_BASE_URL: https://api.example.com # REPLACE
        run: |
          npm ci
          npm run build
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${vars.AWS_DEPLOY_ROLE_ARN}
          aws-region: ${env.AWS_REGION}
      - name: Upload to S3 and invalidate CloudFront
        env:
          BUCKET: ${vars.FRONTEND_BUCKET}
```

.github/workflows/deploy-ec2.yml

```
DISTRIBUTION_ID: ${vars.CLOUDFRONT_DISTRIBUTION_ID}
run: |
  aws s3 sync "$WEB_DIR/dist" "s3://$BUCKET" --delete \
    --exclude index.html --cache-control "public,max-age=31536000,immutable"
  aws s3 cp "$WEB_DIR/dist/index.html" "s3://$BUCKET/index.html" \
    --cache-control "no-cache"
  aws cloudfront create-invalidation --distribution-id "$DISTRIBUTION_ID" \
    --paths "/index.html"
```

F · Rendered JSON references (not committed; shown for learning)

These are what AWS stores after Terraform renders the HCL. Do not maintain them alongside the .tf files.

ecs-task-definition.json — Equivalent of ecs-task-definition.tf.

ecs-task-definition.json reference/ecs-task-definition.json

```
{
  "family": "ai-lab-prod-api",
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "1024",
  "memory": "2048",
  "runtimePlatform": { "operatingSystemFamily": "LINUX", "cpuArchitecture": "X86_64" },
  "executionRoleArn": "arn:aws:iam::111122223333:role/ai-lab-prod-api-execution-role",
  "taskRoleArn": "arn:aws:iam::111122223333:role/ai-lab-prod-api-task-role",
  "containerDefinitions": [
    {
      "name": "api",
      "image": "111122223333.dkr.ecr.us-east-1.amazonaws.com/ai-lab/api:0123abc",
      "essential": true,
      "portMappings": [{ "containerPort": 8080, "protocol": "tcp" }],
      "environment": [
        { "name": "SERVER_PORT", "value": "8080" },
        { "name": "POSTGRES_HOST", "value": "ai-lab-prod-db.abcdefghijk.us-east-1.rds.amazonaws.com" },
        { "name": "POSTGRES_PORT", "value": "5432" },
        { "name": "POSTGRES_DB", "value": "ai_lab" }
      ],
      "secrets": [
        { "name": "POSTGRES_USER", "valueFrom": "arn:aws:secretsmanager:us-east-1:111122223333:secret:rds!db-1a2b3c-AbCdEf:username::" },
        { "name": "POSTGRES_PASSWORD", "valueFrom": "arn:aws:secretsmanager:us-east-1:111122223333:secret:rds!db-1a2b3c-AbCdEf:password::" },
        { "name": "OPENAI_API_KEY", "valueFrom": "arn:aws:secretsmanager:us-east-1:111122223333:secret:ai-lab/prod/api-GhIjKl:OPENAI_API_KEY::" }
      ],
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -fsS http://localhost:8080/actuator/health || exit 1"],
        "interval": 30, "timeout": 5, "retries": 3, "startPeriod": 120
      },
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/ai-lab-prod-api",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "api"
        }
      },
      "stopTimeout": 30
    }
  ]
}
```

IAM: GitHub OIDC trust policy — From github-oidc.tf.

IAM: GitHub OIDC trust policy reference/iam-github-trust-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::111122223333:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
          "token.actions.githubusercontent.com:sub": "repo:OWNER/ai-engineering-lab:environment:production"
        }
      }
    }
  ]
}
```

IAM: ecs-tasks trust policy — From iam.tf (ECS).

IAM: ecs-tasks trust policy [reference/iam-ecs-tasks-trust-policy.json](#)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ecs-tasks.amazonaws.com" },
      "Action": "sts:AssumeRole",
      "Condition": { "StringEquals": { "aws:SourceAccount": "111122223333" } }
    }
  ]
}
```

IAM: execution role secrets policy — From iam.tf (ECS).

IAM: execution role secrets policy [reference/iam-ecs-execution-secrets-policy.json](#)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadInjectedSecrets",
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource": [
        "arn:aws:secretsmanager:us-east-1:111122223333:secret:rds!db-1a2b3c-AbCdEf",
        "arn:aws:secretsmanager:us-east-1:111122223333:secret:ai-lab/prod/api-GhIjKl"
      ]
    }
  ]
}
```

app-secret.example.json — Shape of ai-lab/prod/api (never commit real values).

app-secret.example.json [reference/app-secret.example.json](#)

```
{
  "OPENAI_API_KEY": "REPLACE-or-empty-string"
}
```